

动态 DP



前置知识：矩阵，树链剖分。

动态 DP 问题是猫锟在 WC2018 讲的黑科技，一般用来解决树上的带有点权（边权）修改操作的 DP 问题。

例子

以这道模板题为例子讲解一下动态 DP 的过程。

例题 洛谷 P4719 【模板】动态 DP



给定一棵 n 个点的树，点带点权。有 m 次操作，每次操作给定 x, y 表示修改点 x 的权值为 y 。你需要在每次操作之后求出这棵树的最大权独立集的权值大小。

广义矩阵乘法 ¶

定义广义矩阵乘法 $A \times B = C$ 为：



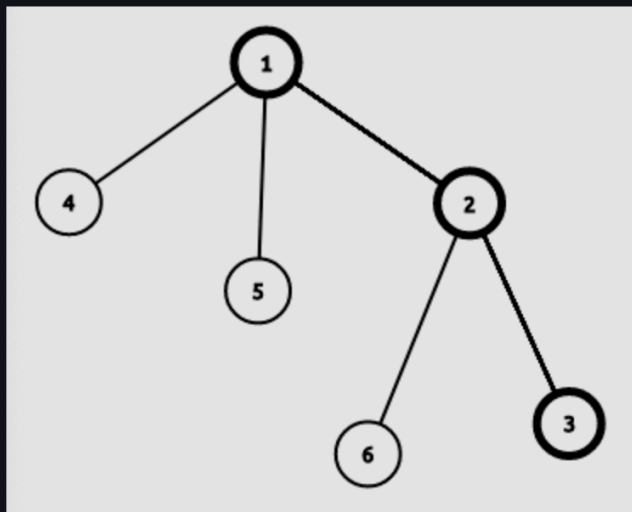
$$C_{i,j} = \max_{k=1}^n (A_{i,k} + B_{k,j})$$

相当于将普通的矩阵乘法中的乘变为加，加变为 $\max$ 操作。

同时广义矩阵乘法满足结合律，所以可以使用矩阵快速幂。

带修改操作

首先将这棵树进行树链剖分，假设有这样一条重链：



设 $g_{i,0}$ 表示不选择 i 且只允许选择 i 的轻儿子所在子树的最大答案， $g_{i,1}$ 表示不考虑 son_i 的情况下选择 i 的最大答案， son_i 表示 i 的重儿子。

假设我们已知 $g_{i,0/1}$ 那么有 DP 方程：

$$\begin{cases} f_{i,0} = g_{i,0} + \max(f_{son_i,0}, f_{son_i,1}) \\ f_{i,1} = g_{i,1} + f_{son_i,0} \end{cases}$$

答案是 $\max(f_{root,0}, f_{root,1})$ 。

可以构造出矩阵：

$$\begin{bmatrix} g_{i,0} & g_{i,0} \\ g_{i,1} & -\infty \end{bmatrix} \times \begin{bmatrix} f_{son_i,0} \\ f_{son_i,1} \end{bmatrix} = \begin{bmatrix} f_{i,0} \\ f_{i,1} \end{bmatrix}$$

注意，我们这里使用的是广义乘法规则。

可以发现，修改操作时只需要修改 $g_{i,1}$ 和每条往上的重链即可。

具体思路

1. DFS 预处理求出 $f_{i,0/1}$ 和 $g_{i,0/1}$ 。
2. 对这棵树进行树链剖分（注意，因为我们对一个点进行询问需要计算从该点到该点所在的重链末尾的区间矩阵乘，所以对于每一个点记录 End_i 表示 i 所在的重链末尾节点编号），每一条重链建立线段树，线段树维护 g 矩阵和 g 矩阵区间乘积。
3. 修改时首先修改 $g_{i,1}$ 和线段树中 i 节点的矩阵，计算 top_i 矩阵的变化量，修改到 fa_{top_i} 矩阵。
4. 查询时就是 1 到其所所在的重链末尾的区间乘，最后取一个 $\max$ 即可。

习题

- SPOJ GSS3 - Can you answer these queries III

```

#include <algorithm>
#include <cstring>
#include <iostream>

using namespace std;

constexpr int MAXN = 500010;
constexpr int INF = 0x3f3f3f3f;

int Begin[MAXN], Next[MAXN], To[MAXN], e, n, m;
int sz[MAXN], son[MAXN], top[MAXN], fa[MAXN], dis[MAXN], p[MAXN], id[MAXN],
    End[MAXN];
// p[i]表示i树剖后的编号, id[p[i]] = i
int cnt, tot, a[MAXN], f[MAXN][2];

struct matrix {
    int g[2][2];

    matrix() { memset(g, 0, sizeof(g)); }

    matrix operator*(const matrix &b) const // 重载矩阵乘
    {
        matrix c;
        for (int i = 0; i <= 1; i++)
            for (int j = 0; j <= 1; j++)
                for (int k = 0; k <= 1; k++)
                    c.g[i][j] = max(c.g[i][j], g[i][k] + b.g[k][j]);
        return c;
    }
} Tree[MAXN], g[MAXN]; // Tree[]是建出来的线段树, g[]是维护的每个点的矩阵

void PushUp(int root) { Tree[root] = Tree[root << 1] * Tree[root << 1 | 1]; }

void Build(int root, int l, int r) {
    if (l == r) {
        Tree[root] = g[id[l]];
        return;
    }
    int Mid = (l + r) >> 1;
    Build(root << 1, l, Mid);
    Build(root << 1 | 1, Mid + 1, r);
    PushUp(root);
}

```

```

matrix Query(int root, int l, int r, int L, int R) {
    if (L <= l && r <= R) return Tree[root];
    int Mid = (l + r) >> 1;
    if (R <= Mid) return Query(root << 1, l, Mid, L, R);
    if (Mid < L) return Query(root << 1 | 1, Mid + 1, r, L, R);
    return Query(root << 1, l, Mid, L, R) *
           Query(root << 1 | 1, Mid + 1, r, L, R);
    // 注意查询操作的书写
}

void Modify(int root, int l, int r, int pos) {
    if (l == r) {
        Tree[root] = g[id[l]];
        return;
    }
    int Mid = (l + r) >> 1;
    if (pos <= Mid)
        Modify(root << 1, l, Mid, pos);
    else
        Modify(root << 1 | 1, Mid + 1, r, pos);
    PushUp(root);
}

void Update(int x, int val) {
    g[x].g[1][0] += val - a[x];
    a[x] = val;
    // 首先修改x的g矩阵
    while (x) {
        matrix last = Query(1, 1, n, p[top[x]], End[top[x]]);
        // 查询top[x]的原本g矩阵
        Modify(1, 1, n,
              p[x]); // 进行修改(x点的g矩阵已经进行修改但线段树上的未进行修改)
        matrix now = Query(1, 1, n, p[top[x]], End[top[x]]);
        // 查询top[x]的新g矩阵
        x = fa[top[x]];
        g[x].g[0][0] +=
            max(now.g[0][0], now.g[1][0]) - max(last.g[0][0], last.g[1][0]);
        g[x].g[0][1] = g[x].g[0][0];
        g[x].g[1][0] += now.g[0][0] - last.g[0][0];
        // 根据变化量修改fa[top[x]]的g矩阵
    }
}

void add(int u, int v) {
    To[++e] = v;
    Next[e] = Begin[u];
    Begin[u] = e;
}

void DFS1(int u) {
    sz[u] = 1;
    int Max = 0;
    f[u][1] = a[u];
    for (int i = Begin[u]; i; i = Next[i]) {

```

```

    int v = To[i];
    if (v == fa[u]) continue;
    dis[v] = dis[u] + 1;
    fa[v] = u;
    DFS1(v);
    sz[u] += sz[v];
    if (sz[v] > Max) {
        Max = sz[v];
        son[u] = v;
    }
    f[u][1] += f[v][0];
    f[u][0] += max(f[v][0], f[v][1]);
    // DFS1过程中同时求出f[i][0/1]
}
}

void DFS2(int u, int t) {
    top[u] = t;
    p[u] = ++cnt;
    id[cnt] = u;
    End[t] = cnt;
    g[u].g[1][0] = a[u];
    g[u].g[1][1] = -INF;
    if (!son[u]) return;
    DFS2(son[u], t);
    for (int i = Begin[u]; i; i = Next[i]) {
        int v = To[i];
        if (v == fa[u] || v == son[u]) continue;
        DFS2(v, v);
        g[u].g[0][0] += max(f[v][0], f[v][1]);
        g[u].g[1][0] += f[v][0];
        // g矩阵根据f[i][0/1]求出
    }
    g[u].g[0][1] = g[u].g[0][0];
}

int main() {
    cin.tie(nullptr) -> sync_with_stdio(false);
    cin >> n >> m;
    for (int i = 1; i <= n; i++) cin >> a[i];
    for (int i = 1; i <= n - 1; i++) {
        int u, v;
        cin >> u >> v;
        add(u, v);
        add(v, u);
    }
    dis[1] = 1;
    DFS1(1);
    DFS2(1, 1);
    Build(1, 1, n);
    for (int i = 1; i <= m; i++) {
        int x, val;
        cin >> x >> val;
        Update(x, val);
    }
}

```

```
matrix ans = Query(1, 1, n, 1, End[1]); // 查询1所在重链的矩阵乘
cout << max(ans.g[0][0], ans.g[1][0]) << '\n';
}
return 0;
}
```

Problem G. 转身

Input file: standard input
Output file: standard output
Time limit: 1 second
Memory limit: 256 megabytes

在一条数轴上有 n 个人，每个人面朝左（'L'）或右（'R'）。给定一个长度为 n 的字符串 S ，其中每个字符是 'L' 或 'R'，表示每个人的初始方向。在每个时刻，当两个相邻的人面对面时（即，左边的人面朝右，右边的人面朝左），他们会同时转身：如果一个人面朝左，他会转向右；反之亦然。

你的任务是确定在没有人再改变方向之后所需的时间。

此外，还有 q 次修改。每次修改给出一个索引 x ，意味着第 x 个人的初始方向被翻转（从 'L' 变为 'R' 或反之）。在每次修改后，输出上述问题的答案。

Input

第一行包含两个整数 n ($1 \leq n \leq 2 \cdot 10^5$) 和 q ($1 \leq q \leq 2 \cdot 10^5$)。
第二行包含一个长度为 n 的字符串 S ，表示每个人的初始方向。
接下来的 q 行包含一个整数 x ($1 \leq x \leq n$)。

Output

输出 q 行。每行包含一个整数，表示答案。

Examples

standard input	standard output
5 5	1
LRLRL	2
5	0
1	3
3	3
4	
3	
5 5	0
RLLLL	3
1	3
2	3
3	0
4	
5	

[AC.]

牛客竞赛
AC.NOWCODER.COM

G - Turn Around

题意

- 给定一个长度为 n ，只包含'L'和'R'的字符串 S ，每次操作同时翻转所有的"RL"子串，问可以进行多少次操作。
- 另外给定 q 次操作，每次把一个位置'L'变成'R'或者把'R'变成'L'，问修改后上述问题的答案。
- $1 \leq n, q \leq 2 \cdot 10^5$ 。

AC.NOWCODER.COM

G - Turn Around

[AC.]

牛客竞赛
AC.NOWCODER.COM

G - Turn Around

题解

- 先把字符串开头的'L'去掉，然后从左到右考虑每个'L'。设 dp_i 表示第 i 个'L'复位所需要的时间； num_i 表示第 i 个'L'的前面有多少个'R'。
- 第 i 个'L'复位的时候有两种情况：
- 某个时刻左侧紧贴另一个'L'，需要等左侧的'L'先走，自己才能动，时间为 $dp_{i-1} + 1$ 。
- 左侧一直紧贴'R'，时间为 num_i 。
- 转移即为 $dp_i = \max(dp_{i-1} + 1, num_i)$ 。

AC.NOWCODER.COM

G - Turn Around

[AC.]

牛客竞赛
AC.NOWCODER.COM

G - Turn Around

题解 (cont'd)

- 用广义矩阵乘法描述转移。设状态向量为 (dp_i, num_i) ，那么：
- 若当前字母为'L'，转移矩阵为 $\begin{pmatrix} 1 & -\infty \\ 0 & 0 \end{pmatrix}$
- 若当前字母为'R'，转移矩阵为 $\begin{pmatrix} 0 & -\infty \end{pmatrix}$

■ 数据结构维护修改，复杂度 $O(n + q \log n)$ 。

AC.NOWCODER.COM

G - Turn Around

```

#include<bits/stdc++.h>
#ifdef LWY
#include "debug.h"
#else
#define debug(...) 0
#endif
using namespace std;
#define ll long long
#define endl '\n'
#define PII pair<int,int>
#define PLL pair<ll,ll>
// #define int long long
const ll mod1=1e9+7,mod2=998244353;
const int INF = 1e9, N = 2e5 + 10;
//不开long long 见祖宗!!!!
struct node
{
    array<array<int, 2>, 2> x;
    node() : x{array<int,2>{0, -INF}, array<int,2>{-INF, 0}} {}
    node(int x1, int x2, int x3, int x4): x({array<int,2>{x1, x2},
array<int,2>{x3, x4}}) {};
};
node operator+(const node xk, const node yk){
    node res;
    for (int i = 0; i < 2; i++)
    {
        for (int j = 0; j < 2; j++)
        {
            for (int k = 0; k < 2; k++)
            {
                res.x[i][j] = max(res.x[i][j], xk.x[i][k] + yk.x[k][j]);
            }
        }
    }
    return res;
}
node char_node(char x)
{
    if (x == 'L') return node(1, -INF, 0, 0);
    return node(0, -INF, -INF, 1);
}
void print(int L, int R, node res)
{
    cerr << L << "~" << R << " : " << endl;
    if (res.x[1][0] < 0) cerr << "R" << endl;
}

```



```

    else cerr << "L" << endl;
    cerr << res.x[0][0] << " " << res.x[0][1] << endl;
    cerr << res.x[1][0] << " " << res.x[1][1] << endl;
    cerr << endl;
}
vector<node> tr(N << 2), a(N);
int ls(int p){ return p << 1;}
int rs(int p){ return p << 1 | 1;}
void push_up(int p)
{
    tr[p] = tr[ls(p)] + tr[rs(p)];
}
void build(int p, int L, int R)
{
    if (L == R)
    {
        tr[p] = a[L];
        // print(L, R, a[L]);
        return;
    }
    int mid = (L + R) / 2;
    build(ls(p), L, mid);
    build(rs(p), mid + 1, R);
    push_up(p);
}
void update(int p, int L, int R, int id, node &val)
{
    if (L == R)
    {
        tr[p] = val;
        return;
    }
    int mid = (L + R) / 2;
    if (id <= mid)
    {
        update(ls(p), L, mid, id, val);
    }
    else
    {
        update(rs(p), mid + 1, R, id, val);
    }
    push_up(p);
}
node query(int p, int pl, int pr, int L, int R)
{
    if (R < pl || L > pr) return node();
    if (pl <= L && R <= pr) return tr[p];
    int mid = (pl + pr) / 2;
    node res = query(ls(p), pl, mid, L, R) + query(rs(p), mid + 1, pr, L,
R);
    return res;
}
void solve()
{

```

```

int n, q;
cin >> n >> q;
set<int> idR;
string s;
cin >> s;
for (int i = 1; i <= n; i++)
{
    if (s[i - 1] == 'R') idR.insert(i);
    a[i] = char_node(s[i - 1]);
}
node Lk = char_node('L');
node Rk = char_node('R');
build(1, 1, n);
while (q--)
{
    // cerr << s << endl;
    int id;
    cin >> id;
    if (s[id - 1] == 'R')
    {
        idR.erase(id);
        s[id - 1] = 'L';
        update(1, 1, n, id, Lk);
    }
    else
    {
        idR.insert(id);
        s[id - 1] = 'R';
        update(1, 1, n, id, Rk);
    }
    if (idR.size() == n || idR.size() == 0) cout << 0 << endl;
    else
    {
        int k = *idR.begin();
        if (k == n - idR.size() + 1) cout << 0 << endl;
        else
        {
            auto res = query(1, 1, n, k, n);
            // print(k, n, res);
            cout << res.x[1][0] << endl;
            // cout << max(max(res.x[0][0], res.x[1][0]), max(res.x[0]
[1], res.x[1][1])) << endl;
        }
    }
    // cerr << s << endl;
    // for (int i = 1; i <= n; i++)
    // {
    //     print(i, i, query(1, 1, n, i, i));
    // }
}
}

signed main(){
    ios::sync_with_stdio(false);
    cin.tie(0), cout.tie(0);
}

```

```
int t = 1;  
// cin >> t;  
while(t--) solve();  
return 0;  
}
```